

透過 Cloud Shell 建構及啟動 Spring Boot Java 應用程式

來源：<https://codelabs.developers.google.com/codelabs/cloud-springboot-cloudshell?hl=zh-tw>

步驟數：6

瞭解如何透過 Cloud Shell 建立及啟動 Spring Boot Java 應用程式，而不必離開瀏覽器。

目錄

1. [1. 事前準備](#)
2. [2. 設定和需求](#)
3. [3. 初始化新的 Spring Boot 應用程式](#)
4. [4. 使用程式碼編輯器建立新的 RESTful 服務](#)
5. [5. 在本機執行應用程式](#)
6. [6. 恭喜](#)

1. 事前準備

[Cloud Shell](#) 是以瀏覽器為基礎的指令列工具，可讓您存取 Google Cloud 資源。您不必在系統中安裝 Cloud SDK 和其他工具，即可輕鬆管理 Cloud Console 專案和資源。

您將使用 Cloud Shell 建構及啟動 Spring Boot 應用程式，完全不必離開瀏覽器。如要這麼做，請使用「[使用 Spring Boot 建構應用程式](#)」中的程式碼範例。

必要條件

- 熟悉 Java 程式設計語言和工具
- 瞭解標準 Linux 文字編輯器，例如 Vim、Emacs 和 nano

執行步驟

- 使用 Cloud Shell。
- 在 Cloud Shell 中建立簡單的 Spring Boot 應用程式。
- 使用 Cloud Shell 中的程式碼編輯器編輯應用程式。
- 從 Cloud Shell 啟動應用程式。

軟硬體需求

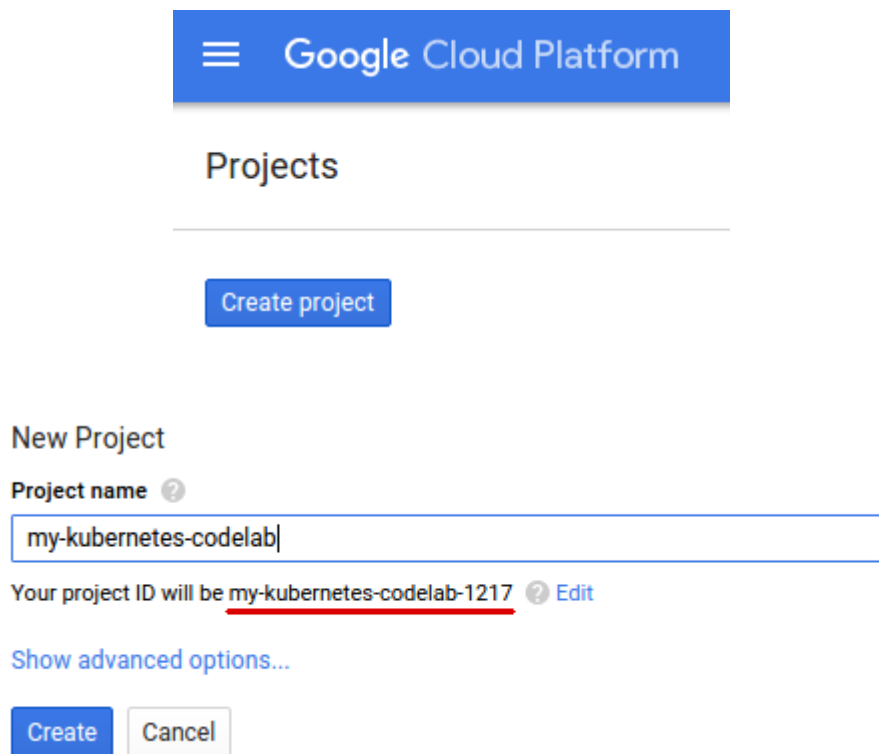
- 具備 Google Cloud 專案
 - 瀏覽器，例如 [Google Chrome](#)
-

步驟 2 · 約 1 分鐘

2. 設定和需求

自修實驗室環境設定

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。登入 [Cloud 控制台](#) 並建立新專案。



請記下專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (圖片中的名稱已遭占用，因此不適用於您，抱歉！)。這個名稱會在後續步驟中稱為 `PROJECT_ID`。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。完成程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行，則可能會增加費用。

Google Cloud 新使用者享有價值 **\$300 美元**的[免費試用期](#)。

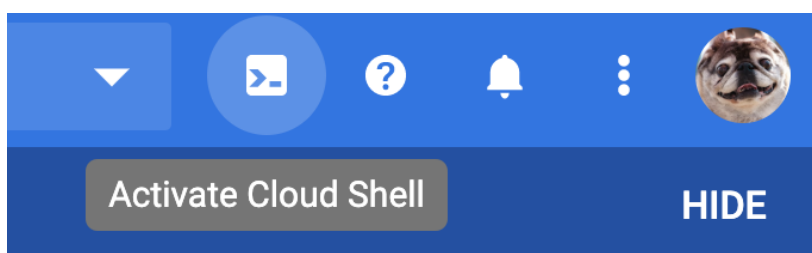
Cloud Shell

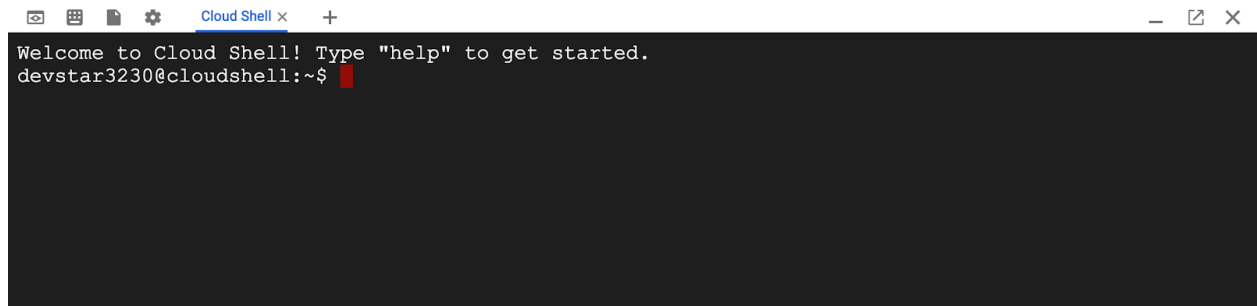
這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。





連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

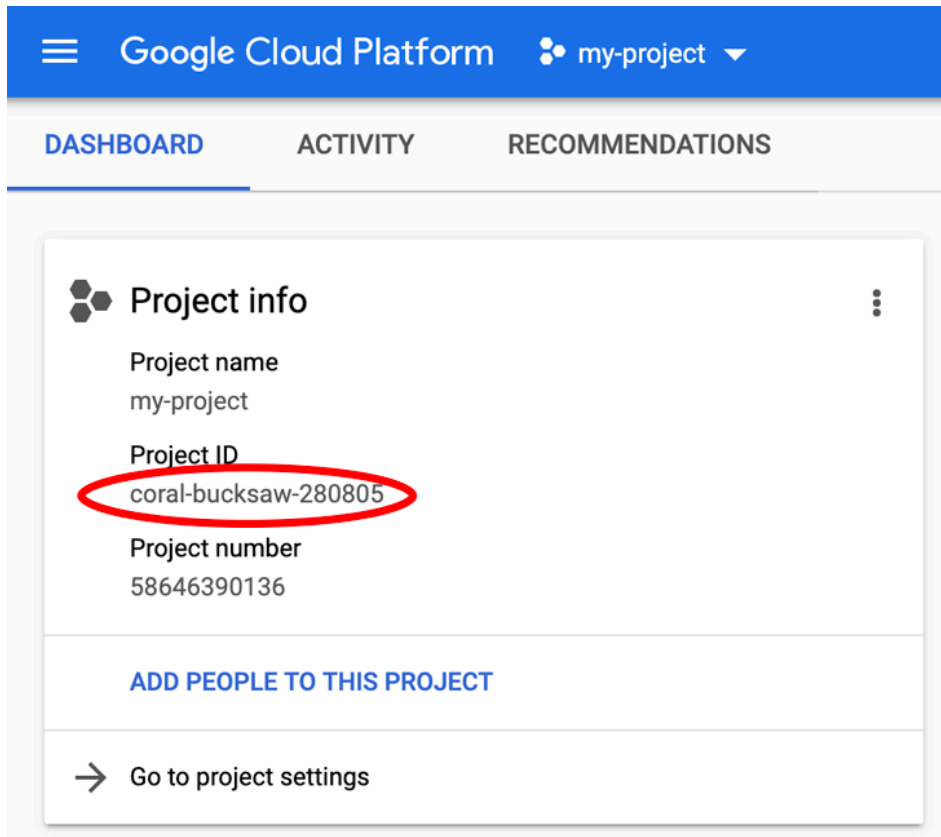
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

步驟 3 · 約 2 分鐘

3. 初始化新的 Spring Boot 應用程式

安裝 Spring Boot 指令列工具後，即可初始化及啟動新的「Hello, World」網頁應用程式。

```
$ curl https://start.spring.io/starter.tgz \  
-d dependencies=web \  
-d type=maven-project \  
-d baseDir=helloworld | tar -xzvf -
```

這會建立新的目錄和 Maven 專案，以及 Maven 的 `pom.xml`、Maven 封裝容器和應用程式進入點。

步驟 4 · 約 5 分鐘

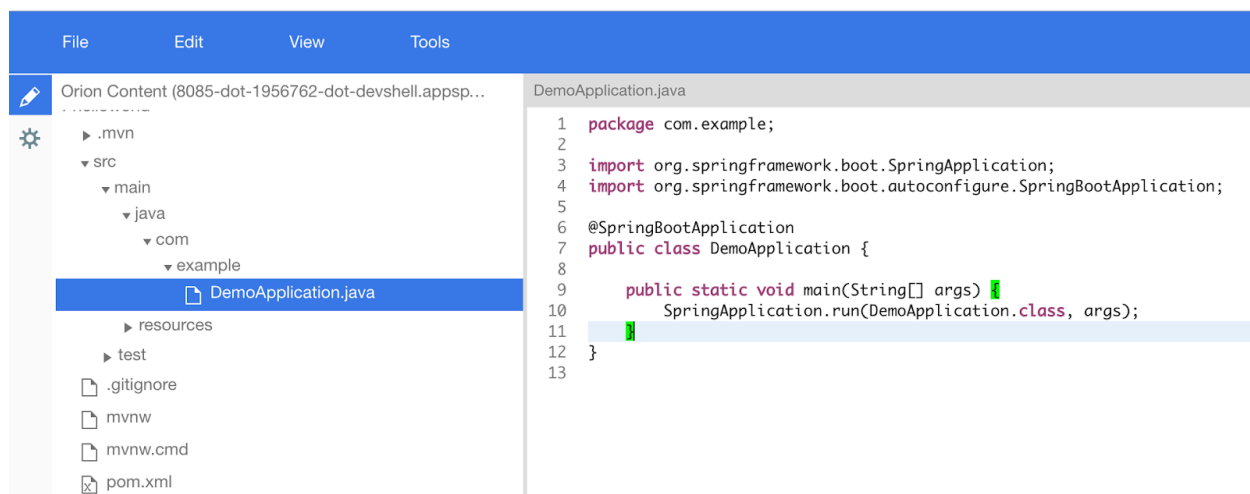
4. 使用程式碼編輯器建立新的 RESTful 服務

1. 按一下 Cloud Shell 選單中的「開啟編輯器」



，開啟程式碼編輯器。

2. 開啟編輯器後，找出 `helloworld/src/main/java/com/example/demo/DemoApplication.java` 檔案。



3. 程式碼開啟後，請建立新的 RESTful 控制器來回應 `Hello`。在 `DemoApplication.java` 檔案中，除了目前的 `HelloWorld` 類別定義外，再新增一個。

`src/main/java/com/example/demo/DemoApplication.java`

```
package com.example;

...

// Add the import
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
public class DemoApplication {
    ...
}

// Add the controller
@RestController
class HelloWorld {
    @GetMapping("/")
    public String greet() {
        return "Hello!";
    }
}
```


4. 儲存檔案！

步驟 5 · 約 2 分鐘

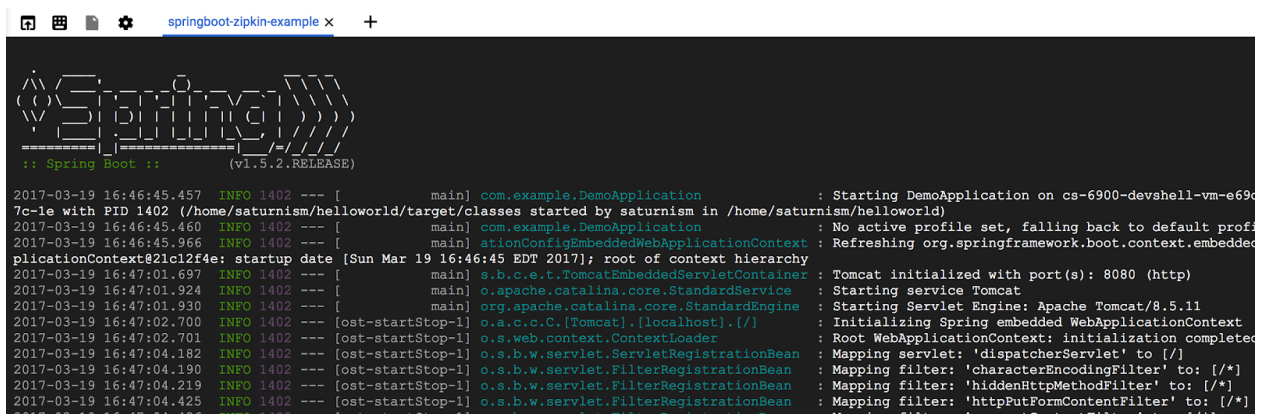
5. 在本機執行應用程式

1. 在程式碼編輯器中開啟終端機，方法是選取 `Terminal`，然後從編輯器選單中選取 `New Terminal`。
2. 確認 `JAVA_HOME` 已設為正確的 JDK 版本：

```
$ export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64/
```

3. 您可以使用 Spring Boot 外掛程式正常啟動 Spring Boot 應用程式。

```
$ cd $HOME/helloworld
$ ./mvnw -DskipTests spring-boot:run
```

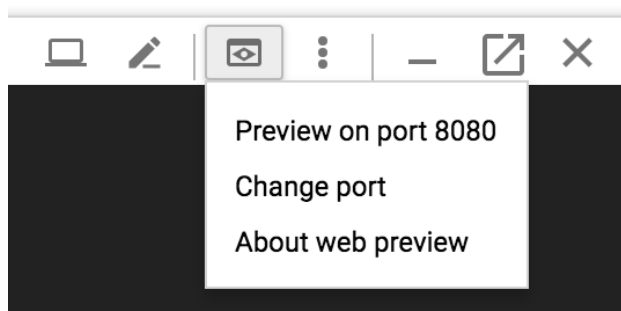


```
springboot-zipkin-example x +
:: Spring Boot ::
2017-03-19 16:46:45.457 INFO 1402 --- [main] com.example.DemoApplication : Starting DemoApplication on cs-6900-devshell-vm-e690
7c-1e with PID 1402 (/home/saturnism/helloworld/target/classes started by saturnism in /home/saturnism/helloworld)
2017-03-19 16:46:45.460 INFO 1402 --- [main] com.example.DemoApplication : No active profile set, falling back to default proff
2017-03-19 16:46:45.966 INFO 1402 --- [main] ationConfigEmbeddedWebApplicationContext : Refreshing org.springframework.boot.context.embedded
plicationContext@21c12f4e: startup date [Sun Mar 19 16:46:45 EDT 2017]; root of context hierarchy
2017-03-19 16:47:01.697 INFO 1402 --- [main] s.b.c.e.t.TomcatEmbeddedServletContainer : Tomcat initialized with port(s): 8080 (http)
2017-03-19 16:47:01.924 INFO 1402 --- [main] o.apache.catalina.core.StandardService : Starting service Tomcat
2017-03-19 16:47:01.930 INFO 1402 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet Engine: Apache Tomcat/8.5.11
2017-03-19 16:47:02.700 INFO 1402 --- [ost-startStop-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2017-03-19 16:47:02.701 INFO 1402 --- [ost-startStop-1] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed
2017-03-19 16:47:04.182 INFO 1402 --- [ost-startStop-1] o.s.b.w.servlet.ServletRegistrationBean : Mapping servlet: 'dispatcherServlet' to [/]
2017-03-19 16:47:04.190 INFO 1402 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean : Mapping filter: 'characterEncodingFilter' to: [/]
2017-03-19 16:47:04.219 INFO 1402 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean : Mapping filter: 'hiddenHttpMethodFilter' to: [/]
2017-03-19 16:47:04.425 INFO 1402 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean : Mapping filter: 'httpPutFormContentFilter' to: [/]
2017-03-19 16:47:04.425 INFO 1402 --- [ost-startStop-1] o.s.b.w.servlet.FilterRegistrationBean : Mapping filter: 'requestContextFilter' to: [/]
```

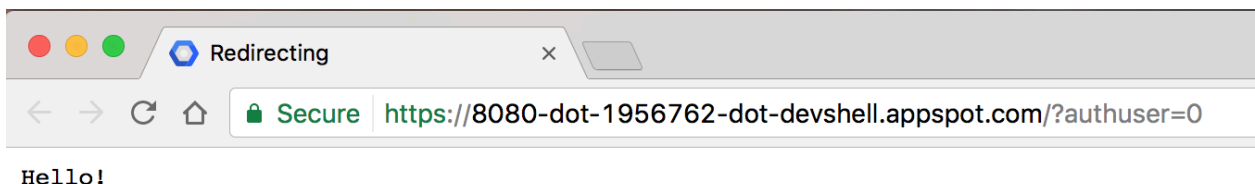
4. 應用程式啟動後，請在 Cloud Shell 工具列中依序點選「Web Preview」（網頁預覽）



和「Preview on port 8080」（透過以下通訊埠預覽：8080）。



瀏覽器會開啟分頁，並連線至您啟動的伺服器。



步驟 6 · 約 0 分鐘

6. 恭喜

您已學會如何直接從 Cloud Shell 建構及啟動新的 Spring Boot Java 網頁應用程式。

瞭解詳情

- [在 Google Cloud 中使用 Java](#)
 - [Java 程式碼範例](#)
-